

STEMKit: A Client-Side Web Architecture for Secure, Reproducible Computational Chemistry and Data Analysis Workflows

Olanrewaju M. Daramola^{*1}

¹Independent Researcher

July 23, 2026

Abstract

Computational chemistry generates a steady stream of small analytical tasks that fall between the major simulation packages: reading a GROMACS `.xvg` file to check convergence, converting coordinates between nm and Å, computing a centre of mass, testing whether two sets of replicates differ, or composing a scheduler script that requests appropriate resources. Each is individually simple, yet collectively they impose enough friction — an environment to configure, a dependency to install, a script to relocate — that they are frequently performed manually and therefore inconsistently. The available remedies each carry a cost: local Python tooling demands an environment that managed institutional machines often disallow; web services require uploading data, which is unacceptable for unpublished results or material under confidentiality agreement; and graphical desktop applications resist scripting and hence reproduction.

We present STEMKit, a suite of 21 browser-based tools built on `@stemkit/core`, a JavaScript library of parsing and numerical routines carrying no dependency on the document object model. All computation executes within the user’s browser using the `FileReader` API; no data is transmitted, no account is required, and no installation is performed. Because all computation runs locally in the browser, no user data reaches an external server. The same modules also execute under Node.js, so an analysis explored interactively can be captured as a script, version-pinned, and placed under continuous integration without rewriting.

The library comprises 15 domain modules validated by 975 automated tests. Numerical results are verified against independent references — SciPy for inferential statistics, NumPy for regression, `scipy.constants` for each conversion factor, and physical invariants for structural geometry — rather than against the implementation itself. Extraction of the computational core from the original monolithic scripts exposed three numerical defects that had propagated into reported output, each of which is documented, corrected, and covered by regression tests.

Keywords: computational chemistry, molecular dynamics, GROMACS, PLUMED, client-side computation, data privacy, reproducibility, open-source software

1 Introduction

The software landscape of computational chemistry is well served at its extremes. Simulation engines such as GROMACS [1] and LAMMPS [2] are mature, extensively validated, and actively maintained; analysis

^{*}ORCID: 0009-0006-3327-2047

frameworks such as MDAnalysis [4] provide comprehensive programmatic access to trajectory data. Between these poles lies a substantial volume of routine work that neither category serves comfortably.

Consider the interval between completing a simulation and reporting it. A researcher must inspect the RMSD to confirm equilibration, extract a radius of gyration, convert a structure between the nanometre convention of GROMACS and the Å convention of the Protein Data Bank, compare replicate measurements statistically, digitise a figure from a paper for comparison, abbreviate journal titles in a reference list, and compose a scheduler script requesting an appropriate resource allocation. None of these operations is intellectually demanding. All of them are error-prone when performed by hand, and all of them are performed repeatedly.

1.1 The friction problem

Three classes of solution exist, and each imposes a characteristic cost.

Local scripting. A Python environment with MDAnalysis, NumPy, and SciPy addresses every task listed above and is the correct instrument for sustained analytical work. Its cost is environmental. Managed institutional workstations frequently prohibit package installation; conda environments drift; and a script written six months ago may no longer execute against the current interpreter. For a researcher whose primary expertise lies in chemistry rather than software engineering, this cost is not negligible.

Web services. Server-side tools eliminate installation entirely but require data transmission. For unpublished results, material subject to a confidentiality agreement, or work under embargo, this is frequently prohibited outright. Even where permitted, the transfer creates a record whose retention policy the researcher does not control.

Desktop applications. Graphical applications avoid both problems but introduce a third: they are difficult to script. An analysis performed by clicking cannot readily be re-executed, version-controlled, or subjected to continuous integration. In practice such an analysis is difficult to reproduce, even when every step was recorded at the time.

1.2 The client-side position

Modern browsers provide a capable computational environment. The `FileReader` API grants access to local files without transmission; JavaScript engines execute numerical code fast enough for the data volumes involved. ECMAScript modules make it possible to organise that code properly, instead of accumulating scripts.

We built STEMKit around these constraints. No server participates in the computation, so there is nothing to which data could be transmitted; users do not have to take a privacy policy on trust. Its reproducibility property derives from a separate architectural decision — the numerical core is extracted into modules with no dependency on the browser, so the identical code that executes behind the interface also executes under Node.js.

This paper describes that architecture, the workflows it supports, the numerical methods underlying each computation, and the defects that the extraction process exposed in the original implementation.

2 Software Architecture

2.1 Separation of computation from presentation

The original implementation followed the pattern common to browser tooling: each of the 21 tools consisted of an HTML document paired with a single JavaScript file combining event handling, rendering, and computation. This arrangement is expedient during development and progressively obstructive thereafter. Computational logic entangled with DOM manipulation cannot be tested without a browser, cannot be reused across tools, and cannot be invoked from a script.

The refactoring applied the *strangler fig* pattern: for each tool, the mathematical and parsing logic was extracted into a pure module beneath `src/core/`, a test suite was written against the extracted module, and the original file was then reduced to DOM wiring that imports the new functions. The user-visible interface is unchanged; the computation beneath it is now independently addressable.

Table 1 enumerates the resulting modules.

Table 1: Modules of `@stemkit/core`. Vendored dependencies are supplied by injection (Section 2.2); modules marked *none* operate without any third-party code.

Module	Domain	Tests	Dependency
bibtex	Reference deduplication, sanitising	89	bibtex-parse-js
statistics	Inferential statistics	82	jStat
structure	Molecular geometry, PDB/GRO/XYZ	79	none
xvg-parser	GROMACS/PLUMED trajectory data	77	none
curve-fitting	Least-squares regression	72	regression.js
plumed	PLUMED input generation	72	none
selection	Atom selection, spatial queries	67	none
slurm	HPC job-script generation	61	none
latex	Table generation and escaping	59	none
error-bars	Group summaries, error bars	57	jStat
units	Physical unit conversion	56	none
journals	Journal title abbreviation	53	none
data-cleaning	Tabular transformation	51	Papa Parse
outliers	Anomaly detection	51	jStat
digitizer	Figure digitisation	49	none
Total		975	

2.2 Dependency injection across execution environments

Four third-party libraries are vendored within the repository: jStat for statistical distributions, Papa Parse for delimited-text parsing, regression.js for curve fitting, and bibtex-parse-js for reference parsing. Each is distributed as a Universal Module Definition (UMD) bundle, which detects its host at run time and either assigns `module.exports` under CommonJS or attaches a global to `window` in a browser.

Neither path is reachable from a plain ECMAScript module. A UMD file contains no `export` statements, so a direct import yields no bindings; and `import { createRequire } from 'module'` — the conventional Node.js workaround — constitutes a hard *resolution* failure in a browser, which occurs before any code executes and therefore cannot be intercepted by `try/catch`.

Rather than branch on the execution environment inside every module, which would render the core untestable in one of its two targets, the library declares the libraries it requires and each host registers them at initialisation:

```
// Node.js: the vendored bundles are loaded through createRequire.
import { createRequire } from 'module';
import { registerVendor } from '@stemkit/core';

const require = createRequire(import.meta.url);
registerVendor({ jStat: require('./js/dependencies/jstat.min.js') });

// Browser: the UMD <script> tags have already installed their globals.
import { registerFromGlobals } from './src/core/index.js';
registerFromGlobals();
```

Listing 1: Registration under Node.js and in the browser.

Modules request their dependencies lazily, at the point of use rather than at import time, so that importing any module never fails merely because an unrelated library has not yet been registered. The result is a single code path shared by both environments, with the vendored libraries remaining the sole source of truth for the numerics in each.

One consequence merits explicit note, as it is a common failure mode. A repository declaring "type": "module" instructs Node.js to parse every .js file beneath it as an ECMAScript module, including the vendored UMD bundles. Under that interpretation the UMD factory takes its browser branch and fails. Scoping "type": "commonjs" to the dependency directory alone restores correct behaviour.

2.3 Testing strategy

The suite comprises 975 tests across the 15 domain modules. Its governing principle is that numerical results are validated against *independent* references rather than against the implementation under test, since a test written from the same source as the code confirms only internal consistency.

Reference values were obtained from SciPy [5] for t-tests, ANOVA, correlation, non-parametric tests, quantiles, and multiple-comparison correction; from NumPy [6] for regression coefficients and descriptive statistics; from `scipy.constants` for every unit conversion factor; and from physical invariants for structural geometry — specifically the molecular weight and centre of mass of water, the orthonormality of rotation matrices, the preservation of interatomic distances under rotation, and round-trip fidelity through every supported file format.

An end-to-end smoke test additionally exercises one representative path through each module against a real installation, detecting the class of failure that unit tests cannot: a broken aggregate export, a misconfigured module type declaration, or a vendored bundle that fails to load.

3 Workflows

3.1 Molecular dynamics trajectory analysis

GROMACS writes analysis output in the Grace format, a plain-text convention in which lines beginning with # are comments, lines beginning with @ are formatting directives, and all remaining non-empty lines are whitespace-delimited numeric records. The directives carry the plot title, axis labels, and per-series legends. Discarding them — as a naive parser that skips all non-numeric lines would — loses precisely the metadata that identifies which column contains which observable.

The `xvg-parser` module recovers this metadata alongside the numeric matrix:

```
import { parseXvg, extractColumn, columnStats } from '@stemkit/core';
import { readFileSync } from 'fs';
```

```
const { matrix, headers, title } = parseXvg(readFileSync('rmsd.xvg', 'utf8'));
// title    -> "RMSD & Radius of Gyration"
// headers -> ["Time (ps)", "Backbone RMSD", "Rg"]

headers.slice(1).forEach((name, i) => {
  const s = columnStats(extractColumn(matrix, i + 1));
  console.log(`${name}: ${s.mean.toFixed(4)} +/- ${s.std.toFixed(4)} nm`);
});
```

Listing 2: Parsing a GROMACS RMSD trajectory.

Malformed records are counted and reported, not silently discarded, so a truncated trajectory is visible to the user instead of quietly shortening the analysis. Two further details of the format are handled explicitly. The Grace multi-set separator & is treated as a delimiter, not parsed as data. Records containing NaN or Infinity are rejected whole; one non-finite entry would otherwise contaminate every downstream statistic.

3.2 PLUMED collective-variable output

PLUMED [3] writes COLVAR files whose header lines begin with #! and carry field names and metadata. Although the convention differs from Grace, the comment character coincides, and the same parser therefore handles both formats without modification — the header lines are skipped as comments and the numeric records parsed identically. This is verified by an explicit test rather than left to assumption.

3.3 Structure files and molecular geometry

The `structure` module parses the three coordinate formats in routine use for classical simulation: PDB and GROMACS `.gro`, both fixed-column formats in which fields are located by character position, and XYZ, which is whitespace-delimited. Fixed-column parsing is essential rather than pedantic: a residue name may legitimately be blank, and a whitespace-splitting parser would silently shift every subsequent field on such a line.

Unit handling is explicit throughout. PDB and XYZ express coordinates in Å; `.gro` uses nm. Every parse result carries its unit, and conversion is always deliberate. Silently mixing the two is the single most direct route to a structure that is wrong by a factor of ten.

3.3.1 Element inference from atom names

Assigning an element to each atom is a prerequisite for any mass-weighted quantity, and PDB atom names are ambiguous by construction. The name CA denotes the α -carbon in a protein residue and calcium in an ion record. The resolution implemented here proceeds in five stages:

1. An explicit element column, where present, takes priority.
2. A leading digit is a hydrogen-count prefix (1HB, 2HG1) and is stripped before the symbol is read.
3. An unambiguous two-letter metal (FE, ZN, SE) is recognised wherever it appears, including within a host residue such as haem or selenomethionine.
4. Otherwise, a two-letter prefix whose second character is uppercase is a PDB remoteness indicator, and only the first letter is the element: CA in a protein residue is C α .
5. As a final fallback, a two-letter symbol is trusted when the residue shares its name, the convention for monatomic ions.

Stages 2 and 3 correct defects present in the original implementation, which assigned fluorine to haem iron, sulfur to selenomethionine selenium, and no element at all to numeric-prefixed hydrogens. The resolution is validated against 40 atom names drawn from real structures, covering protein backbone and side-chain atoms, nucleic acid atoms, water, monatomic ions, and metal centres.

3.3.2 Geometric quantities

The module computes the geometric centroid, the mass-weighted centre of mass, the axis-aligned bounding box, and the radius of gyration

$$R_g = \sqrt{\frac{\sum_i m_i |\mathbf{r}_i - \mathbf{R}|^2}{\sum_i m_i}}, \quad \mathbf{R} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}, \quad (1)$$

together with rigid-body rotation about an arbitrary pivot using intrinsic Z–Y–X Euler angles. Velocities present in a `.gro` file rotate with the frame but are never translated, so that a round trip through rotation and export preserves the kinetic state.

3.4 HPC job-script generation

Composing a scheduler script correctly requires knowledge that is engine-specific and easily misapplied. Two aspects dominate the resulting waste of allocation.

First, the resource model differs by engine. GROMACS is threaded and requires one MPI rank per node with many CPUs per task; LAMMPS is MPI-parallel and requires many ranks per node. Emitting the wrong shape either idles most of a node or oversubscribes it.

Second, and less widely appreciated, the SLURM `--mem` directive in an array job specifies memory *per task*, not per job. A request of 32 GB across a hundred-task array asks the scheduler for 3.2 TB concurrently. The generator computes the true in-flight total and reports it:

```
import { generateScript } from '@stemkit/core';

const { script, warnings } = generateScript({
  engine: 'gromacs', jobName: 'prod_md', partition: 'gpu',
  nodes: 1, gpus: 1, cpusPerTask: 16,
  walltime: '24:00:00', memory: '32G',
  modules: ['gcc/11.3', 'cuda/12.1', 'gromacs/2023.3'],
  tpr: 'md.tpr', deffnm: 'md', maxh: 23.5
});

warnings.forEach(w => console.warn(`[${w.level}] ${w.message}`));
```

Listing 3: Generating a GROMACS submission script.

Generated scripts include checkpoint-resume logic, so that a requeued job continues from its last checkpoint rather than silently restarting from $t = 0$ — a failure mode that consumes an entire allocation while appearing to succeed. Warnings are returned as structured objects rather than formatted strings, leaving presentation to the caller.

3.5 Figure digitisation

Recovering numerical values from a published figure requires a two-point affine calibration per axis. On a logarithmic axis the interpolation must be performed in logarithmic space,

$$v(p) = 10^{\log_{10} v_1 + \frac{(p - p_1)(\log_{10} v_2 - \log_{10} v_1)}{p_2 - p_1}}, \quad (2)$$

because a decade occupies a constant pixel distance there. Treating a logarithmic axis linearly is the most frequent source of error in figure digitisation, and it produces values that are plausible rather than obviously wrong.

The module additionally reports the data-space uncertainty corresponding to a one-pixel positioning error. Stating this alongside digitised values is honest practice: the precision of a digitised point is determined by the resolution of the figure, not by the number of decimal places the export happens to print.

3.6 Unit conversion

Sixty-four units across ten categories are supported, each defined by its ratio to a category base unit. Conversion therefore reduces to

$$x_{\text{to}} = x_{\text{from}} \cdot \frac{f_{\text{to}}}{f_{\text{from}}}, \quad (3)$$

which keeps the table linear in the number of units rather than quadratic and renders every entry independently checkable against its cited source. Conversion factors are CODATA 2018 values [7]; each was verified against `scipy.constants` during testing. Temperature is handled separately, since the Celsius and Fahrenheit scales are affine rather than multiplicative and cannot be expressed by a single factor.

4 Statistical Rigour

4.1 Test selection and assumption checking

The statistical module implements Welch’s [8] and Student’s t-tests, the paired t-test, one-way ANOVA, Pearson correlation, the Mann–Whitney U test, and the Wilcoxon signed-rank test. Welch’s test is the default for independent samples, as it does not assume equal variances and loses negligible power when they are in fact equal.

Every parametric test reports an effect size and, where standard, a confidence interval. Assumptions are checked and surfaced rather than presumed: normality by the D’Agostino–Pearson K^2 omnibus test [9], and homogeneity of variance by the median-centred Brown–Forsythe variant of Levene’s test [10], chosen for its markedly greater robustness to non-normality than the original mean-centred formulation. Where an assumption fails, a non-parametric alternative is recommended.

4.2 Numerical methods

Table 2 maps each computed quantity to its numerical method and validation reference.

4.3 Cancellation in tail probabilities

Upper-tail probabilities are computed through the complementary form of the incomplete beta and gamma functions rather than as $1 - F(x)$. The two expressions are algebraically identical; numerically they are not. Once $F(x)$ rounds to unity in double precision, the subtraction cancels completely and the result is exactly zero.

The consequence is not academic. For the one-way ANOVA used in our test suite ($F = 377.545$ on 2 and 21 degrees of freedom), the naive form returns $p = 0$, whereas

$$p = I_{\frac{d_2}{d_2+d_1F}} \left(\frac{d_2}{2}, \frac{d_1}{2} \right) = 3.461\,174\,738\,291\,4 \times 10^{-17}, \quad (4)$$

agreeing with SciPy to twelve significant figures. A continuous test statistic cannot produce $p = 0$. When that value appears in published output it indicates a numerical fault in the software, not an unusually strong effect.

Table 2: Numerical methods and validation references.

Quantity	Method	Validated against
Student / Welch t , p	Regularised incomplete beta $I_x(a, b)$	<code>scipy.stats.ttest_ind</code>
Welch–Satterthwaite ν	Closed form	SciPy, hand calculation
One-way ANOVA F , p	Sum-of-squares decomposition; I_x tail	<code>scipy.stats.f_oneway</code>
Pearson r , CI	Fisher z transform	<code>scipy.stats.pearsonr</code>
Mann–Whitney U	Tie-corrected normal approximation	<code>scipy.stats.mannwhitneyu</code>
Wilcoxon W	Signed-rank, zeros discarded	<code>scipy.stats.wilcoxon</code>
D’Agostino–Pearson K^2	Skewness and kurtosis transforms	<code>scipy.stats.normaltest</code>
Levene W	Brown–Forsythe (median-centred)	<code>scipy.stats.levene</code>
Quantiles	Linear interpolation (type 7)	<code>numpy.percentile</code>
Multiple comparisons	Holm–Bonferroni [11]	Independent implementation
Grubbs G	Inverted t critical value	<code>scipy.stats.t.ppf</code>
Modified Z-score	Median/MAD, 0.6745 scaling [12]	<code>numpy</code>
Least-squares fitting	Normal equations (regression.js)	<code>numpy.polyfit</code>

The same treatment is applied to the Student t , χ^2 , and normal tails. Beyond $|z| \approx 8$ the vendored `erfc` implementation underflows, and a documented asymptotic expansion takes over so that a p -value is reported as 1.5×10^{-23} rather than as zero.

4.4 Defects exposed by extraction

Separating computation from presentation subjected the numerical code to independent verification for the first time. Three defects were identified that had propagated into reported output.

Standardised moments. Skewness and kurtosis are defined against the *population* standard deviation,

$$G_1 = \frac{\sqrt{n(n-1)}}{n-2} \cdot \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma_{\text{pop}}} \right)^3, \quad \sigma_{\text{pop}} = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2}. \quad (5)$$

The original implementation used the Bessel-corrected sample standard deviation, deflating skewness by a factor of $((n-1)/n)^{3/2}$ — approximately 15 % at $n = 10$. Because both moments feed the D’Agostino–Pearson statistic, every normality p -value the tool reported was affected.

Tail underflow. As described above, all upper-tail probabilities were computed by subtraction and floored at zero for strong effects.

Element misassignment. Haem iron was assigned the mass of fluorine (18.998 Da rather than 55.845 Da), selenomethionine selenium the mass of sulfur, and numeric-prefixed hydrogens no mass at all. Molecular weights and centres of mass computed for metalloproteins were correspondingly wrong.

Each defect is corrected and covered by a regression test. Users who generated figures with earlier versions of these tools should re-check any reported skewness, normality p -value, or metalloprotein mass.

4.5 An inherited limitation

We have not fixed one limitation, only documented it. The vendored `regression.js` library fits exponential, power, and logarithmic models by *linearisation*: taking logarithms and performing least squares in the

transformed space, additionally weighting the fit by y . This minimises error in logarithmic space rather than in the original units and is therefore not the maximum-likelihood fit under additive Gaussian noise.

The magnitude is material. For a clean doubling series the library returns a growth rate of 0.69022 where unweighted logarithmic least squares gives $\ln 2 = 0.69315$. Neither value is incorrect — they answer different questions — but the difference is large enough to matter when a rate constant is reported. The `fitCurve` function exposes a `linearised` flag so that callers may surface the distinction, and users requiring publication-grade non-linear fits are directed to Levenberg–Marquardt optimisation on untransformed data.

5 Availability and Reproducibility

STEMKit is released under the MIT licence. The browser tools require no installation; the library is obtained by cloning the repository and running `npm install`.

The complete test suite executes with `npm test`, and an end-to-end smoke test with `node tests/smoke.mjs`. Both are intended to be run by users evaluating the software, not solely by its maintainers: a library whose numerical claims cannot be checked by its users offers reproducibility only in principle.

Each tool that generates a figure also emits a standalone matplotlib script reproducing that figure from the original data file. This closes the loop between interactive exploration and reproducible output: a plot examined in the browser can be regenerated offline, unchanged, without the browser.

6 Conclusion

We have described STEMKit, a suite of 21 browser-based tools for computational chemistry built on a tested, dependency-injected JavaScript core. The architecture addresses three constraints simultaneously that existing tooling addresses only in pairs: it requires no installation, transmits no data, and remains scriptable.

The privacy property is structural rather than contractual. Because computation occurs entirely within the browser, there is no server to receive data and consequently no policy to trust. For researchers handling unpublished results or material under confidentiality agreement, this distinction is the difference between a usable tool and an unusable one.

The reproducibility property follows from the separation of computation from presentation. The identical modules that execute behind the browser interface execute under Node.js, so an analysis explored interactively may be captured as a script and version-pinned without rewriting.

The defects found during extraction were the most useful outcome of this work. All three had survived in software that ran without error and produced plausible numbers. None would have been caught by testing the code against our own expectations of it; each surfaced only when the results were compared with SciPy and NumPy. A 15 % error in a reported skewness is not visible by inspection. Nor is a p -value floored at zero, which looks like a very strong result. We would encourage other authors of scientific software to check their numerics against an independent implementation.

Data and Code Availability

Source code, the complete test suite, and documentation are available at <https://github.com/LD-Shell/stemkit> under the MIT licence. The browser tools are hosted at <https://stemkit.net>.

Conflicts of Interest

The author declares no competing financial or non-financial interests.

Acknowledgements

We thank the maintainers of jStat, Papa Parse, regression.js, Plotly, KaTeX, and 3Dmol.js, whose libraries STEMKit builds upon.

References

- [1] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, *SoftwareX* **1–2**, 19–25 (2015). <https://doi.org/10.1016/j.softx.2015.06.001>
- [2] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, and S. J. Plimpton, LAMMPS — a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, *Comput. Phys. Commun.* **271**, 108171 (2022). <https://doi.org/10.1016/j.cpc.2021.108171>
- [3] G. A. Tribello, M. Bonomi, D. Branduardi, C. Camilloni, and G. Bussi, PLUMED 2: New feathers for an old bird, *Comput. Phys. Commun.* **185**(2), 604–613 (2014). <https://doi.org/10.1016/j.cpc.2013.09.018>
- [4] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, and O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, *J. Comput. Chem.* **32**(10), 2319–2327 (2011). <https://doi.org/10.1002/jcc.21787>
- [5] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, *et al.*, SciPy 1.0: fundamental algorithms for scientific computing in Python, *Nat. Methods* **17**(3), 261–272 (2020). <https://doi.org/10.1038/s41592-019-0686-2>
- [6] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, *et al.*, Array programming with NumPy, *Nature* **585**(7825), 357–362 (2020). <https://doi.org/10.1038/s41586-020-2649-2>
- [7] E. Tiesinga, P. J. Mohr, D. B. Newell, and B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, *Rev. Mod. Phys.* **93**(2), 025010 (2021). <https://doi.org/10.1103/RevModPhys.93.025010>
- [8] B. L. Welch, The generalization of ‘Student’s’ problem when several different population variances are involved, *Biometrika* **34**(1–2), 28–35 (1947). <https://doi.org/10.1093/biomet/34.1-2.28>
- [9] R. B. D’Agostino, An omnibus test of normality for moderate and large size samples, *Biometrika* **58**(2), 341–348 (1971). <https://doi.org/10.1093/biomet/58.2.341>
- [10] M. B. Brown and A. B. Forsythe, Robust tests for the equality of variances, *J. Am. Stat. Assoc.* **69**(346), 364–367 (1974). <https://doi.org/10.1080/01621459.1974.10482955>
- [11] S. Holm, A simple sequentially rejective multiple test procedure, *Scand. J. Stat.* **6**(2), 65–70 (1979).
- [12] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers*, ASQC Quality Press, Milwaukee, WI (1993).

- [13] F. E. Grubbs, Procedures for detecting outlying observations in samples, *Technometrics* **11**(1), 1–21 (1969). <https://doi.org/10.1080/00401706.1969.10490657>
- [14] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed., Cambridge University Press, Cambridge (2007).